

CHAPTER 2, PART 2

ArrayList (Impl.) Linked List (Concept, Appl.)

Key topics

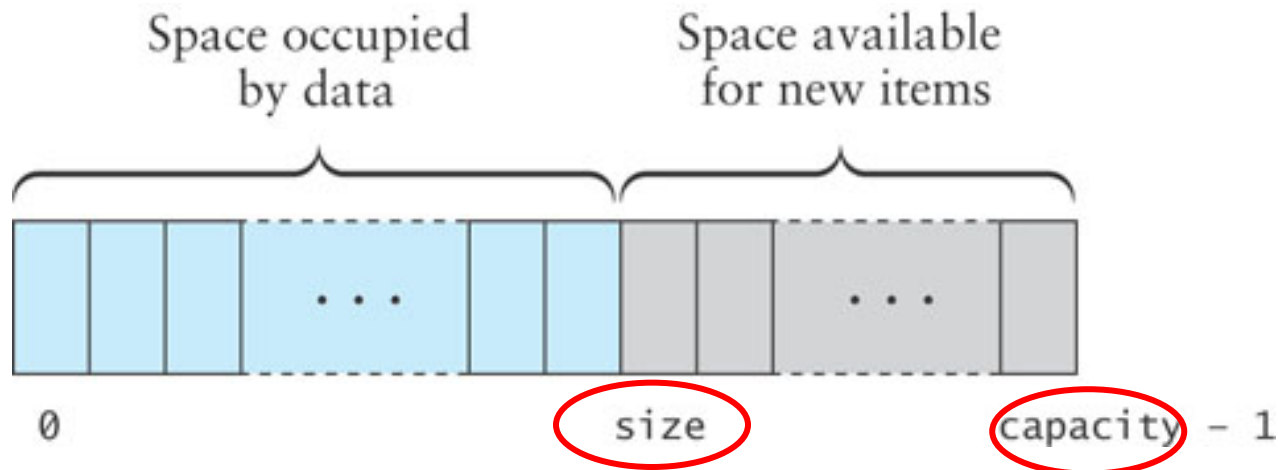
- Array List
 - Implementation
- Linked List
 - ▣ Internal Structure
 - ▣ Use and Application

Implementation of an ArrayList Class

Section 2.4

Implementing an ArrayList Class

- KWArrayList: simple implementation of ArrayList
 - ▣ data field *capacity* - Physical size of array
 - ▣ data field *size* - Actual number of data items



KWArrayList Fields

```
import java.util.*;
// This class implements some of the methods of the Java
//   ArrayList class

public class KWArrayList<E> {
    // Data fields
    /** The default initial capacity */
    private static final int INITIAL_CAPACITY = 10;

    /** The underlying data array */
    private E[] theData;

    /** The current size */
    private int size = 0;

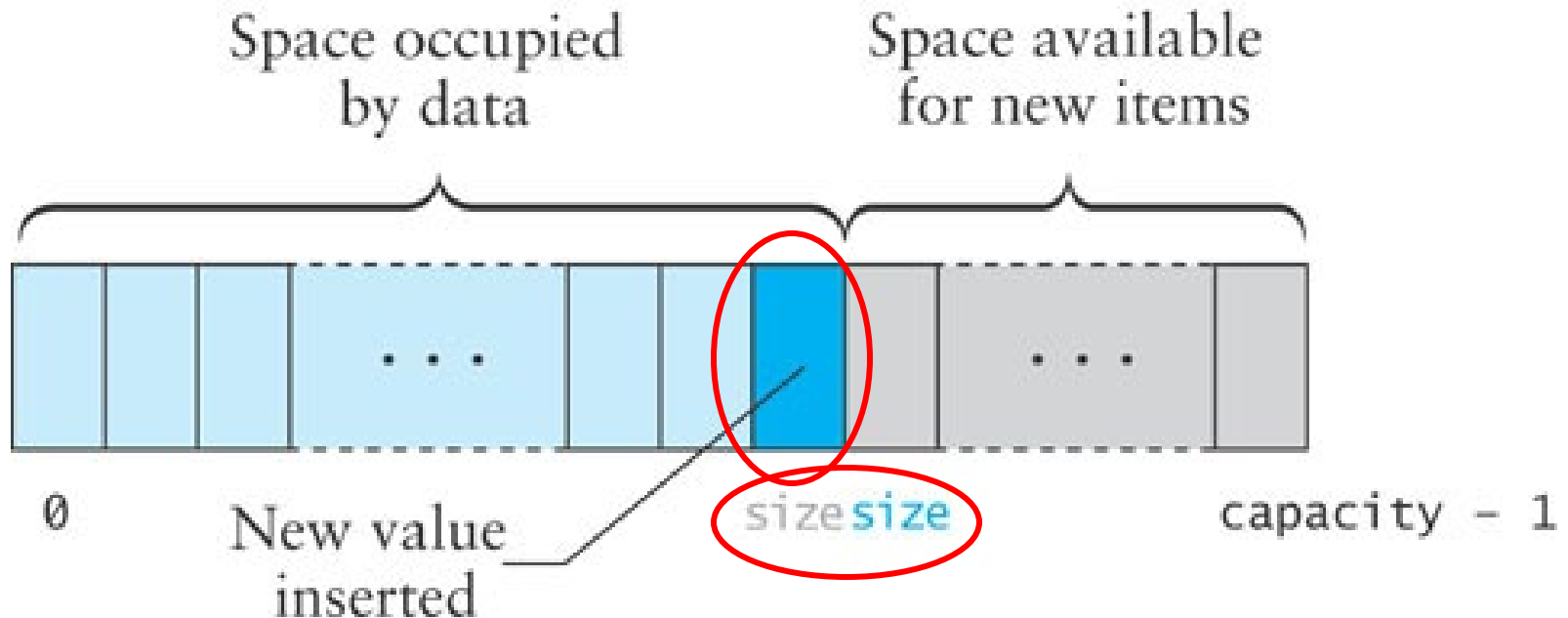
    /** The current capacity */
    private int capacity = 0;
    ...
}
```

KWArrayList Constructor

```
public KWArrayList () {  
    capacity = INITIAL_CAPACITY;  
    theData = (E[]) new Object[capacity];  
}  
  
public KWArrayList (int capacity) {  
    //book exercise  
}
```

Implementing ArrayList - add(E)

- If **size is less than capacity**, then to append a new item
 1. insert the new item at the position indicated by the value of `size`
 2. increment the value of `size`
 3. return `true` to indicate successful insertion



Implementing ArrayList - add(E)

If **size is same as capacity**, grow the array

□ **reallocate Method**

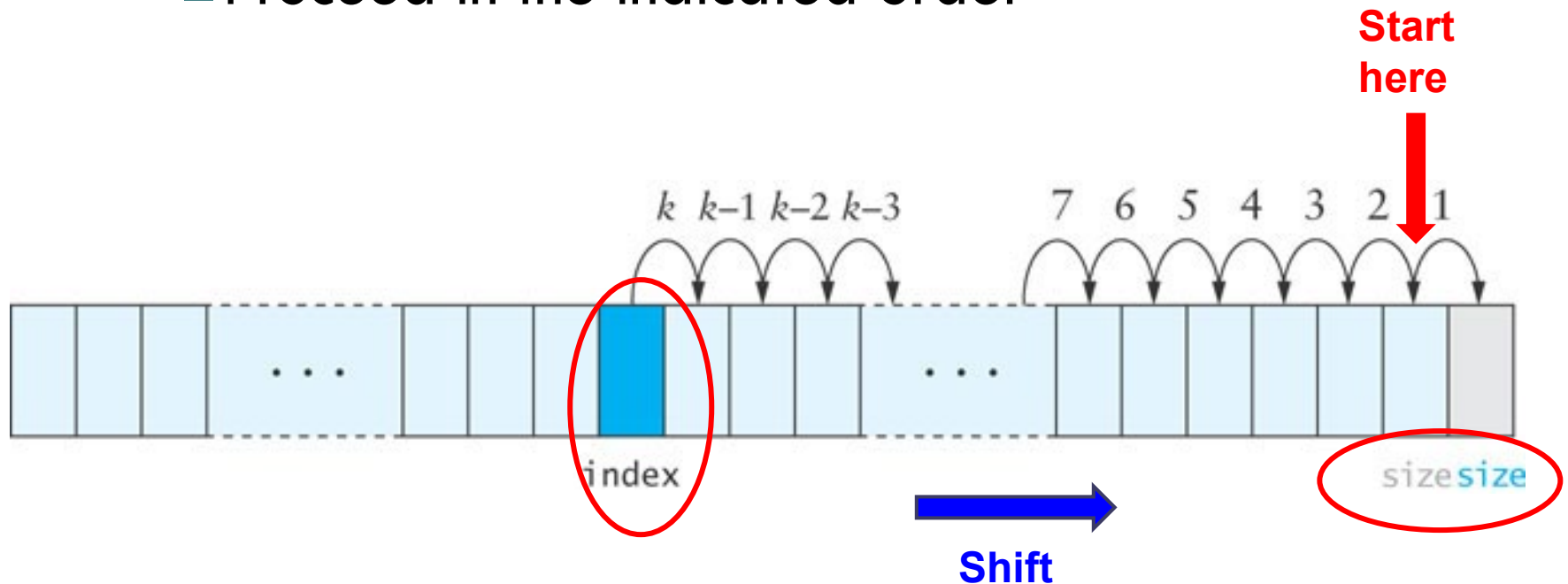
1. Create a **new array** that is **twice the size** of the **current array**
2. **Copy the contents** of the current array to the new array
3. **Use the new array** as the underlying data array

```
private void reallocate () {  
    capacity *= 2; //amortization  
                                //original, new length  
    theData = Arrays.copyOf(theData, capacity);  
}
```


Implementing ArrayList

– `add(int index, E anEntry)`

- To **insert into the middle** of the array,
 - ▣ **Shift over** values at the insertion point to make room,
 - **Begin at the end** of the array
 - Proceed in the indicated order



Implementing ArrayList

– add(int index, E anEntry)

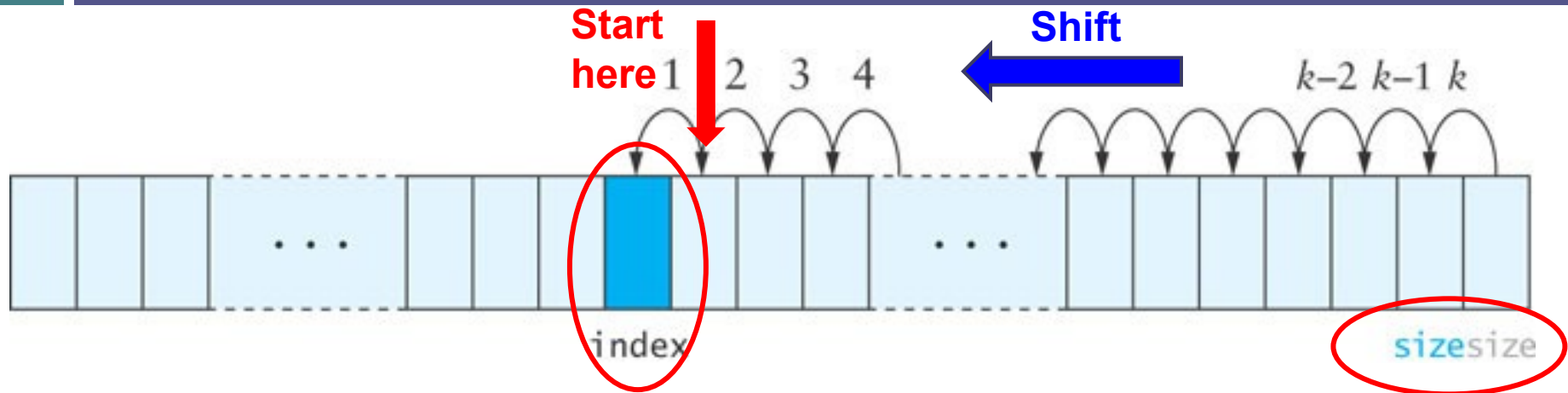
```
public void add (int index, E anEntry) {  
    // check bounds  
    if (index < 0 || index > size) {  
        throw new ArrayIndexOutOfBoundsException(index);  
    }  
  
    // Make sure there is room  
    if (size == capacity) {  
        reallocate();  
    }  
  
    // shift data  
    for (int i = size; i > index; i--) {  
        theData[i] = theData[i-1];  
    }  
  
    // insert item  
    theData[index] = anEntry;  
    size++;  
}
```

Implementing ArrayList – set and get

```
public E get (int index) {
    if (index < 0 || index >= size) {
        throw new ArrayIndexOutOfBoundsException(index);
    }
    return theData[index];
}

public E set (int index, E newValue) {
    if (index < 0 || index >= size) {
        throw new ArrayIndexOutOfBoundsException(index);
    }
    E oldValue = theData[index];
    theData[index] = newValue;
    return oldValue;
}
```

Implementing ArrayList - remove



- To **remove an item in the middle** of the array,
 - ▣ **Shift over item** following the deletion point to close the gap
 - **Begin with the item immediately following the removed element**
 - **Proceed in the indicated order**
- Can **choose to shrink** array capacity during deletion (How?)

Implementing ArrayList

– remove(int index)

```
public E remove (int index) {  
  
    if (index < 0 || index >= size) {  
        throw new ArrayIndexOutOfBoundsException(index);  
    }  
  
    E returnValue = theData[index];  
  
    //shift  
    for (int i = index + 1; i < size; i++) {  
        theData[i-1] = theData[i];  
    }  
  
    size--;  
    return returnValue;  
}
```

Array based list data structure

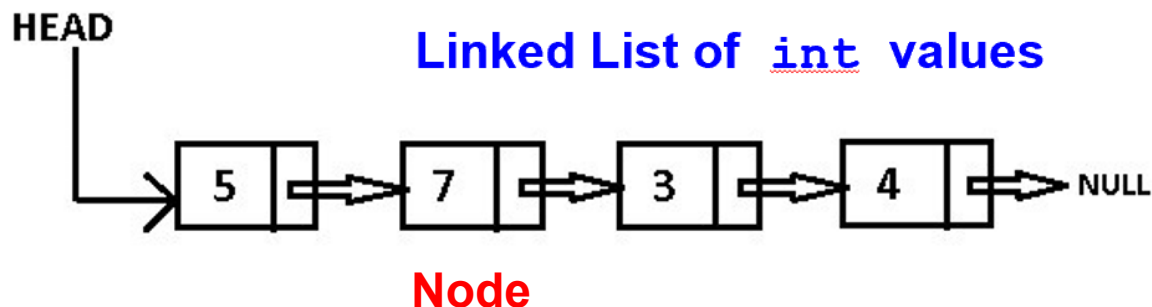
- ❑ Store data in dynamic array (contiguous space)
- ❑ Must shift elements during insertion/deletion in the middle
- ❑ Used when frequently
 - ▣ Add new elements to the end of a list
 - ▣ Randomly retrieve elements based on index
- ❑ Is Cache friendly
- ❑ Used to build advanced cache friendly data structures
- ❑ Not good for frequent insertion/deletion in the middle
- ❑ Example -- Java ArrayList class

Single Linked Lists

Section 2.5

Linked List Data Structure

- ArrayList problem
 - ▣ Shift elements when add and remove in the middle
- Linked lists
 - ▣ Store data items in Nodes at arbitrary locations
 - Typically non-contiguous space
 - ▣ Use links to connect different data items in same collection



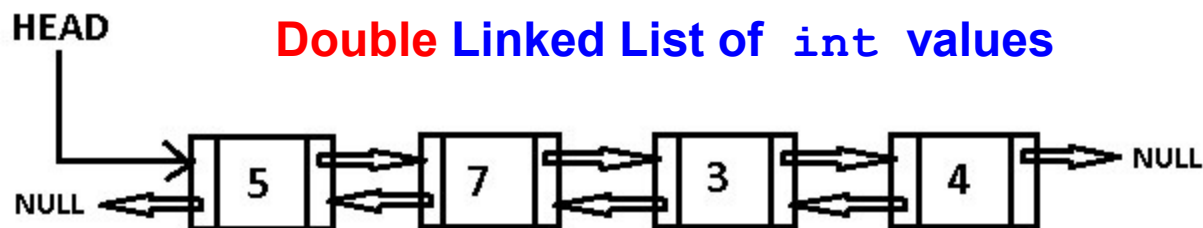
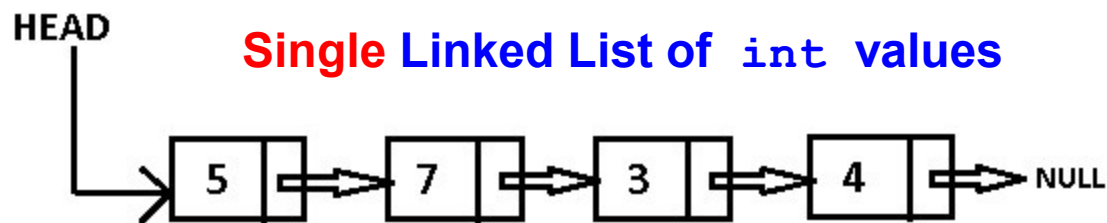
Linked List Data Structure

Single linked lists

- Use 1 link per node to mark immediately following node

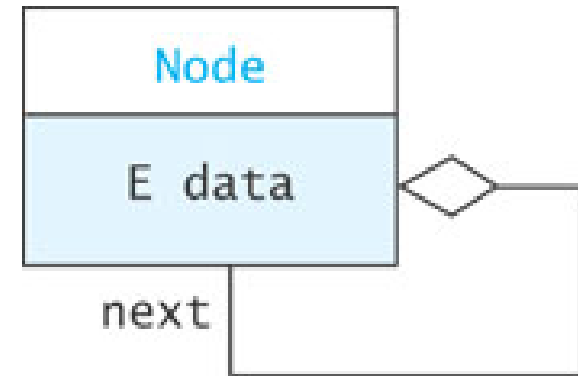
Double linked lists

- Use 2 links per node to mark the immediately following node and immediately preceding node



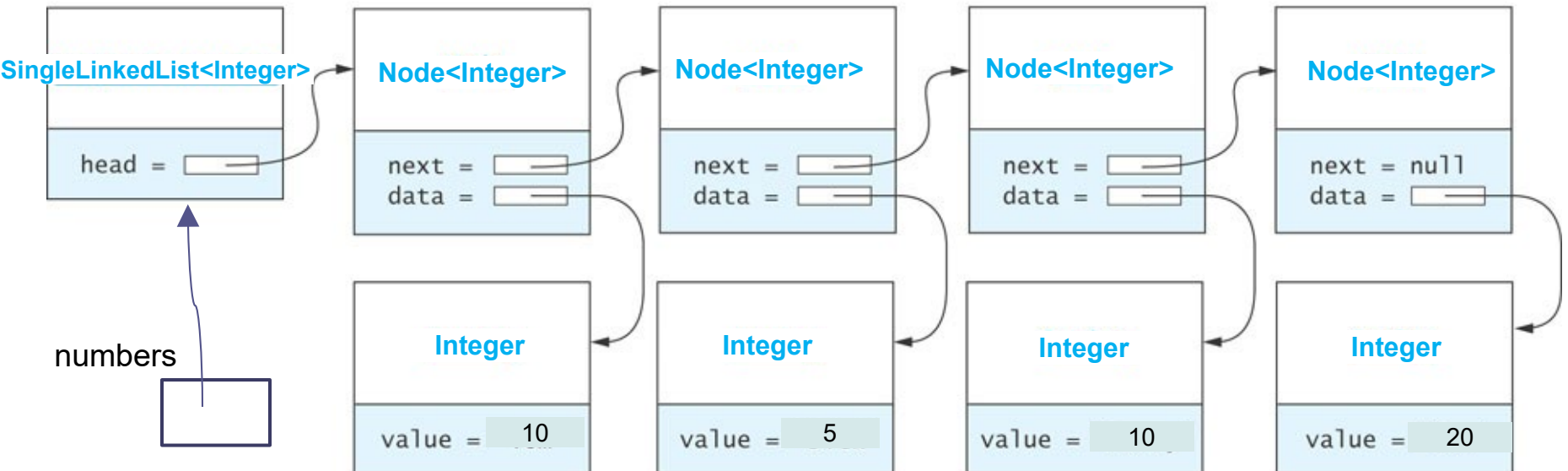
A Linked List Node

- A **node** can contain:
 - ▣ a **data item**
 - ▣ one or more **links**
- A **link** is a **reference to a list node**
- Node in textbook contains
 - ▣ a data field named `data` of type `E`
 - `E` – generic element type
 - `E` can be `Integer`, `String`, `Student` etc
 - ▣ a reference to the next node, named `next`



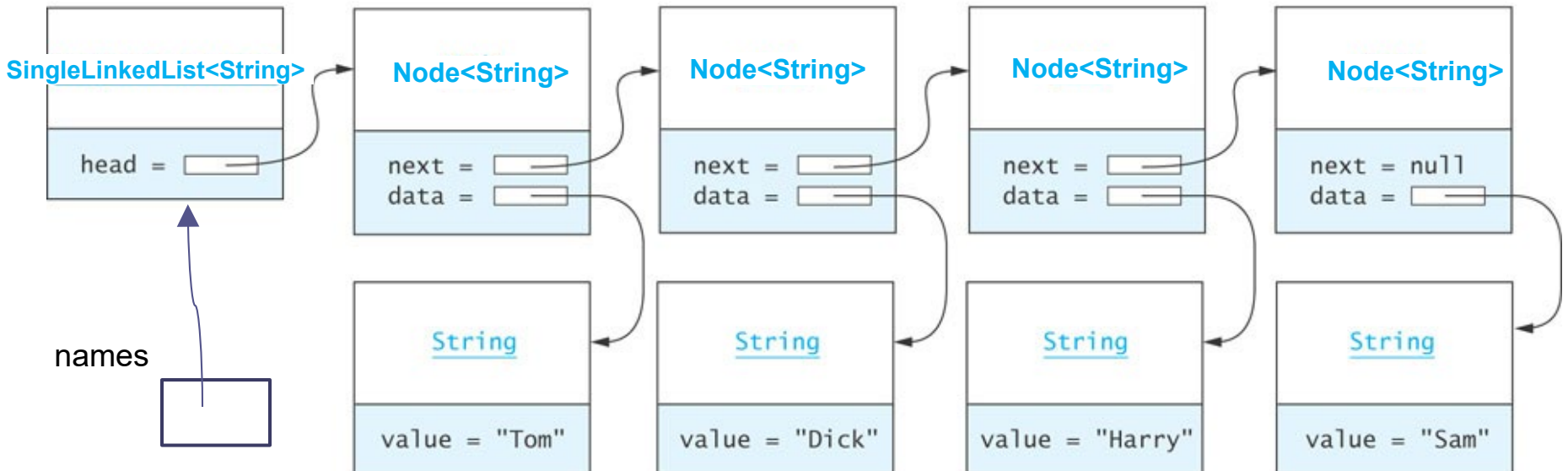
Generic Linked List Data Structure

An example of a single linked list of `Integer` objects
E is `Integer` here.



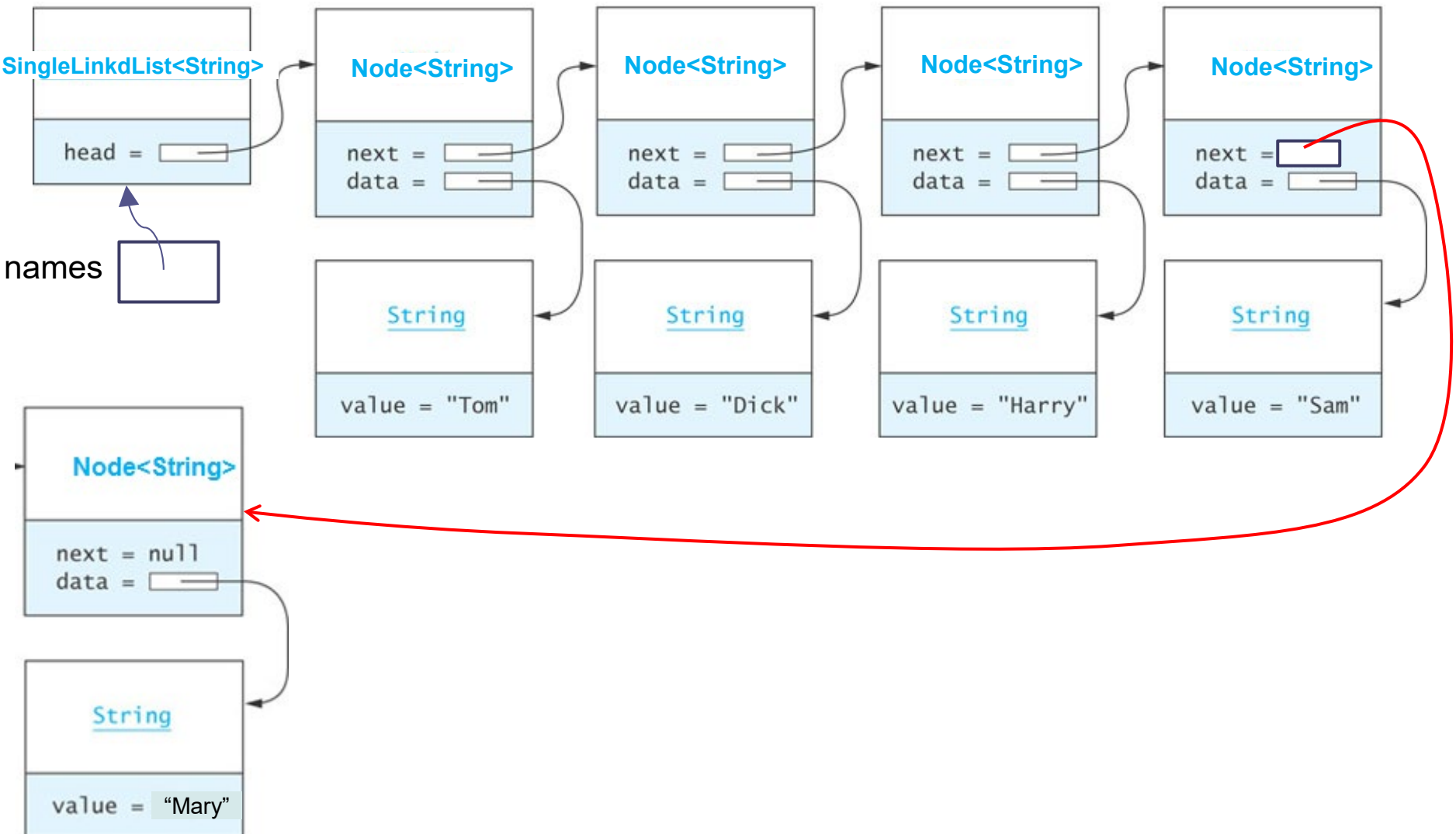
Generic Linked List Data Structure

An example of a single linked list of `String` objects.
E is `String` here.



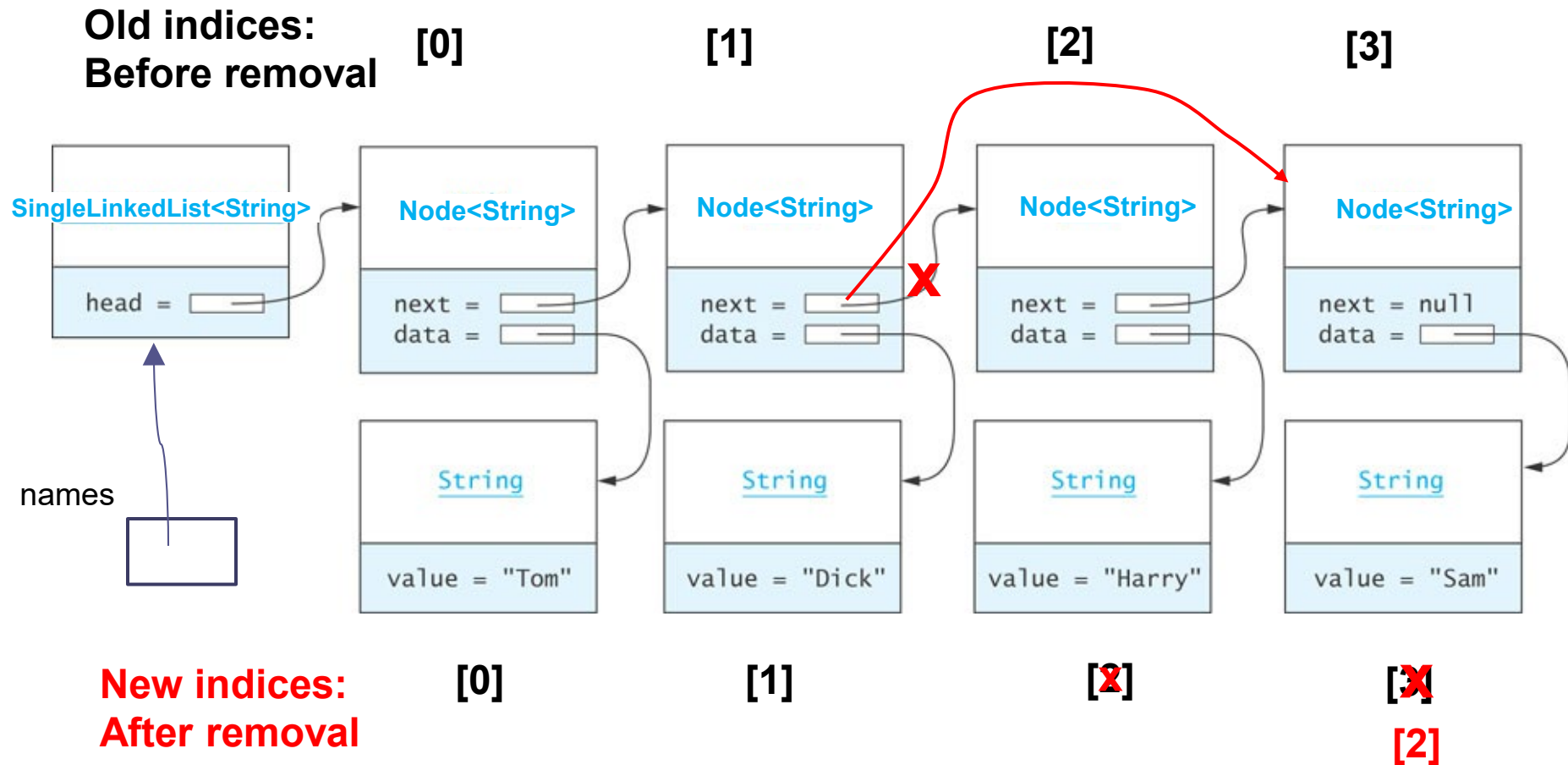
Generic Linked List Data Structure

--Append an element: `names.add("Mary")`



Generic Linked List Data Structure

--Remove an element: `names.remove(2)`



Using Linked Lists - Examples

- Use Java API

- ▣ Interface: `java.util.List`

- <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html>

- ▣ Class: `java.util.LinkedList`

- <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/LinkedList.html>

- Using `LinkedList<E>` class is very similar to using `ArrayList<E>` class because both implement the same interface `List<E>`.

List (Java) – Common Methods

LinkedList, ArrayList

- ❑ `public int size()`
- ❑ `public boolean contains(Object obj)`
- ❑ `public E get(int index)`
- ❑ `public void set(int index, E item)`
- ❑ `public int indexOf(Object obj)`
- ❑ `public boolean add(E item)`
- ❑ `public void add(int index, E item)`
- ❑ `public E remove(int index)`
- ❑ `public boolean remove(Object obj)`

Using Linked Lists - Examples

- LinkedListTest.java, ArrayListTest.java
 - ▣ Basic usage of LinkedList<E> and contrast with ArrayList<E>
 - ▣ Process Linked lists of
 - Integer objects, String objects
 - ▣ Use `java.util.List` **interface**, so that the actual list can be either an array list or a linked list.

Using Linked Lists - Examples

- `LinkedListTest2.java`

- ▣ Create, populate, and sum up every element in a `LinkedList` of `Integer` objects

- `LinkedListTest3.java`

- ▣ Create, populate, and print a `LinkedList` of objects of `Computer` class and its subclasses

- `ManageStudents.java`

- ▣ Create, populate, print, and manipulate a `LinkedList` of objects of `Student` class

Linked List Application

- PhoneDirectory.java

- ▣ Use a `LinkedList` of `DirectoryEntry` objects as the type of its private data field

- PhoneDirectoryTest.java

- ▣ Use `PhoneDirectory` class to create a phone book.